

PHP 类型混淆绕过

PHP 弱类型系统的安全陷阱：当比较语义与实际预期不一致时，认证和校验逻辑就会被绕过。

案例代码

```
<?php
if($_POST[user] && $_POST[pass]) {
    $conn = mysql_connect("*****", "****", "*****");
    mysql_select_db("challenges") or die("Could not select database");
    if ($conn->connect_error) {
        die("Connection failed: " . mysql_error($conn));
    }
}

$user = $_POST[user];
$pass = md5($_POST[pass]);
$sql = "select pwd from interest where uname='$user'";
$query = mysql_query($sql);
if (!$query) {
    printf("Error: %s\n", mysql_error($conn));
    exit();
}

$row = mysql_fetch_array($query, MYSQL_ASSOC);
if (($row[pwd]) && (!strcasecmp($pass, $row[pwd]))) {
    echo "<p>Logged in! Key:*****</p>";
}
else {
    echo("<p>Log in failure!</p>");
}
?>
```

代码问题

1. SQL 注入（最严重）

`$user` 未经任何转义直接拼入 SQL 语句，使用已废弃的 `mysql_*` 扩展（PHP 5.5.0 起弃用，PHP 7.0.0 起移除）：

```
$sql = "select pwd from interest where uname='$user'";
```

注入 payload: ' OR 1=1 -- 使 WHERE 条件恒真，返回第一条记录的密码哈希。

2. 条件作用域错误

只有数据库连接被包裹在 `if($_POST[user] && $_POST[pass])` 内，SQL 查询和认证逻辑在 POST 参数为空时仍会执行——此时 `$conn` 未定义，触发运行时错误。正确做法是整个认证流程都应置于条件内。

3. 裸字符串数组键

```
$row[pwd] // PHP 将 pwd 视为未定义常量，fallback 到字符串 "pwd"，同时产生 E_NOTICE
```

应写为 `$row['pwd']`。虽然此处不影响逻辑，但在复杂代码中可能意外引用到同名常量。

4. strcmp 的隐秘行为（见下文数组绕过）

`strcmp` 在 PHP 5.x 中接收非字符串参数时返回 NULL，而 `!NULL` 为 `true`。

5. md5() 的误用

`md5($_POST['pass'])`：当 `$_POST['pass']` 为数组时（如 `pass[]=foo`），`md5()` 返回 NULL 并产生 Warning（PHP 5.x）或抛出 `TypeError`（PHP 8.x）。MD5 本身也已被密码学攻击（碰撞可在秒级完成），不适合用于密码哈希。

一、== vs ===：PHP 比较的核心陷阱

根本原因

PHP 由 C 实现，继承了 C 语言将非零值视为 `truthy` 的习惯，同时为了 Web 开发的“便利”，在字符串与数字之间做了大量隐式类型转换。`==` 触发**类型转换后再比较**（`type juggling`），`===` 则要求**类型和值都相同**。

PHP 的 `==` 比较规则可概括为：

1. 双方类型相同 → 按各自类型的比较规则
2. 一方为数字（int/float），另一方为字符串 → **字符串转为数字**后再比较
3. 一方为 bool，另一方为非 bool → **非 bool 转为 bool**后再比较
4. 一方为 null，另一方为字符串/数字 → null 转为 `""` 或 `0` 后比较
5. 数组与标量比较 → **数组永远 > 标量**

关键比较表

左操作数	右操作数	== 结果	原因
"123"	123	true	字符串 "123" 转为整数 123
"abc"	0	true	字符串 "abc" 转数字为 0, 0 == 0
"0e123456"	"0e999999"	true	双方都转为科学计数法浮点数 0.0
"admin"	0	true	"admin" → 0, 0 == 0
"admin"	true	true	"admin" → true (非空字符串)
null	false	true	特殊规则
0	false	true	int 0 转为 false
" "	false	true	空字符串转为 false
[]	false	true	空数组转为 false
"1admin"	1	true	字符串从左解析数字直到遇到非数字字符
"0"	false	true	"0" → 0 → false
" "	0	true	空白字符串 → 0

PHP 字符串转数字规则

PHP 的字符串到数字转换不是调用 intval()，而是从左到右解析：

"123abc"	→ 123	(遇到 'a' 停止)
"abc123"	→ 0	(第一个字符就不是数字)
"0e12345"	→ 0.0	(科学计数法: 0 × 10^12345)
"1e2"	→ 100.0	(1 × 10^2)
" 123"	→ 123	(前导空白被忽略)
"0x1A"	→ 0	(十六进制不会被解析)
"1.5"	→ 1.5	(浮点)
"1.5e2"	→ 150.0	(浮点科学计数法)

在认证场景中的后果

```
// 假设数据库密码哈希是某非 0e 开头的值
$stored_hash = "a1b2c3d4e5f6...";

// 攻击者输入密码为 true (bool) 或 0 (int)
// 如果代码用的是 ==
if (md5($_POST['pass']) == $stored_hash) // 可能意外通过
```

但更实际的是下面两个经典绕过技术。

二、数组绕过： strcmp / strcasecmp 的非字符串参数

核心原理

PHP 5.x 中， strcmp() 和 strcasecmp() 接收非字符串参数时会返回 NULL 并产生 Warning：

```
strcmp([], "any_string")    → NULL + Warning
strcmp("any_string", [])    → NULL + Warning
strcasecmp([], "hash")      → NULL + Warning
```

而 PHP 的布尔转换中 NULL 转为 false，因此：

```
!strcmp([], "secret")    → !NULL → !false → true    // 绕过！
!strcasecmp([], "hash") → !NULL → !false → true    // 绕过！
```

触发方式

在 HTTP 请求中将参数以数组形式发送：

```
POST /login.php HTTP/1.1
Content-Type: application/x-www-form-urlencoded

user=admin&pass[]=anything
```

PHP 收到 pass[]=anything 后， \$_POST['pass'] 的值是 ['anything']（数组），而不是字符串 'anything'。

受影响的函数族

函数	数组参数返回值	绕过条件
strcmp(\$a, \$b)	NULL (PHP 5.x)	!strcmp(arr, str) → true
strcasecmp(\$a, \$b)	NULL (PHP 5.x)	!strcasecmp(arr, str) → true
preg_match(\$p, \$a)	false (所有版本)	!preg_match(p, arr) → true（注意：与 0 次匹配的返回值相同）

函数	数组参数返回值	绕过条件
<code>strpos(\$h, \$a)</code>	NULL (PHP 5.x)	<code>strpos(str, arr) === false</code> → 既是"未找到"也是"参数错误"

本案例中的适用性

在案例代码中：

```
$pass = md5($_POST['pass']); // $_POST['pass'] 是数组 → md5() 返回 NULL
// $pass 现在是 NULL (string)
// strcmp(NULL, $row['pwd']) → 将 NULL 视为 "" → 正常比较，不会产生 NULL 返回值
```

所以此处的 `md5()` 充当了"过滤器"——它把数组转成了 NULL，恰好阻止了数组绕过 `strcmp`。但这纯属巧合，并非有意为之的安全措施。

如果代码是 `strcmp($_POST['pass'], $row['pwd'])`（未经 `md5`），数组绕过就成立了。

版本差异

PHP 版本	<code>strcmp([], "str")</code> 行为
PHP 5.x	返回 NULL + Warning
PHP 7.x	返回 NULL + Warning
PHP 8.0+	抛出 TypeError ，不再返回 NULL

PHP 8.0 起严格的类型检查使此类绕过失效，但大量遗留系统仍运行在 PHP 5.x / 7.x 上。

绕过 `preg_match` 的特殊情况

```
// 常见 WAF 模式：检查输入是否包含危险字符
if (preg_match('/[<>"\']/', $_GET['name'])) {
    die("XSS detected");
}

// 攻击者传入 name[]=payload
// preg_match('/[<>"\']/', ['payload']) → false (PHP < 8.0)
// !false → true → 执行 die()
// 看似被拦截，但结果是：
```

```
// - false === false → WAF 误认为"安全" (0 次匹配)
// - 实际上 preg_match 是因为参数类型错误返回的 false
```

正确做法：用 `===` 区分"0 次匹配"和"匹配出错"：

```
$result = preg_match($pattern, $input);
if ($result === false) {
    die("Invalid input type");
}
if ($result > 0) {
    die("Dangerous pattern detected");
}
```

三、MD5 "0e" 魔法哈希

核心原理

PHP 的 `==` 比较两个字符串时，如果两个字符串都**形如数字**，则转为数字再比较。

当一个字符串以 `0e` 开头且后续字符全是数字时，PHP 将其解析为科学计数法浮点数：

```
"0e83040045199349405802421990321" → 0 × 10^83040045199349405802421990321 = 0.0
"0e462097431906509019562988736854" → 0 × 10^462097431906509019562988736854 = 0.0
```

两个不同的字符串转换后都是 `0.0`，因此 `==` 返回 `true`。

利用条件

1. 代码使用 `==`（而非 `===`）比较哈希值
2. 攻击者的输入经 `md5()` 后产生的哈希恰好以 `0e` 开头且全为数字
3. 数据库中存储的哈希也恰好满足同样条件（少见），**或**攻击者利用了 SQL 注入先控制返回的哈希值

更常见的情况是——攻击者无法控制数据库中的哈希，但可以通过 SQL 注入让查询返回一条攻击者已知密码的记录，然后同时提交 `user` 和 `pass`。

两个最经典的 MD5 0e 哈希输入

输入	MD5 哈希
240610708	0e462097431906509019562988736854
QNKCDZO	0e83040045199349405802421990321
s878926199a	0e545993274517709034328855841020
s155964671a	0e342768416822451524974117254469
s214587387a	0e848240448830537924465865611904
s214587387a	0e848240448830537924465865611904
s878926199a	0e545993274517709034328855841020
s1091221200a	0e940624217856561557816327384675
s1885207154a	0e509367213418206700842008763514

SHA-1 同样受影响

输入	SHA-1 哈希
10932435112	0e07766915004133176347055865026311692244
aaroZmOk	0e66507019969427134894567494305185566735

为什么 == 能防御

```
"0e46209743..." == "0e83040045..." // true (都转为 0.0)
"0e46209743..." === "0e83040045..." // false (字符串字面不同)
```

=== 比较字符串时不做类型转换，直接将字符串内容作为比较对象。两个不同的字符串永远不可能 === 全等。

本案例中的适用性

案例代码使用的是 `strcmp($pass, $row['pwd'])` 而非 `==`，因此 **0e 绕过在此不适用**。但这是 PHP 安全审计中的高频考点，看到 `md5(...)` `==` `$hash` 就要警觉。

四、综合防御

问题	防御
SQL 注入	使用 PDO + 参数化查询；永远不拼接用户输入到 SQL
== 类型混淆	涉及安全判断时一律使用 ===

问题	防御
数组绕过 <code>strcmp</code>	比较前用 <code>is_string()</code> 验证参数类型；或升级至 PHP 8.0+
0e 魔法哈希	使用 <code>===</code> 比较哈希；使用 <code>hash_equals()</code> 防止时序攻击
数组绕过 <code>preg_match</code>	用 <code>=== false</code> 检查返回值，区分"出错"和"0 次匹配"
弱哈希算法	使用 <code>password_hash()</code> / <code>password_verify()</code> ， <code>bcrypt/argon2</code>
<code>\$_POST['key']</code> 裸字符串	始终加引号 <code>\$_POST['key']</code> ；设置 <code>error_reporting(E_ALL)</code>

安全比较的标准写法

```
// 字符串比较（防数组绕过）
if (!is_string($input) || !is_string($expected)) {
    die("Invalid input type");
}
if (strcmp($input, $expected) !== 0) { // 注意 !== 而非 ==
    die("Mismatch");
}

// 哈希比较（防 0e 绕过 + 防时序攻击）
if (!hash_equals($computed_hash, $stored_hash)) {
    die("Invalid credentials");
}

// 正则匹配（防数组绕过）
$result = preg_match($pattern, $input);
if ($result === false) {
    die("Invalid input type");
}
if ($result > 0) {
    // 匹配到了
}
```

参考

- PHP Manual: [Comparison Operators](#)
- PHP Manual: [Type Juggling](#)
- OWASP: [PHP Type Juggling](#)
- CWE-597: Use of Wrong Operator in String Comparison

- PHP 8.0 Migration: [Stricter type checks for arithmetic/bitwise operators](#)